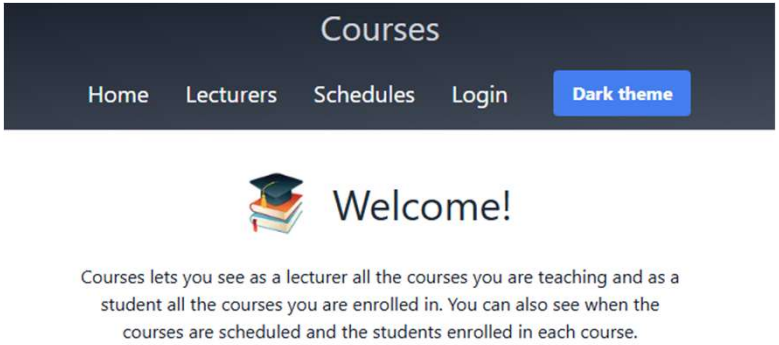
Toegepaste Informatica

Full-stack software development

React Context

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Feature request: light/dark theme switcher



Let’s build this!

Component tree

- Layout (layout.tsx)
  - Root layout defining **<body>** that needs a “dark” or “light” CSS class
    ➔“Theme” state needs to be defined here
- Header (header.tsx)
  - Must display the theme toggle button
- ThemeToggleButton
  - Renders the button with text “Dark theme” of “Light theme”
  - Needs and changes the “theme” state defined in layout.tsx

Layout

```tsx
'use client';

import { useState } from 'react';
import '@styles/globals.css';
import Header from '@components/header';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => {
    setTheme(theme === 'light' ? 'dark' : 'light');
  };

  return (
    <html lang="en">
      <body className={theme}>

        <Header theme={theme} toggleTheme={toggleTheme} />

        <main className="p-6 min-h-screen flex flex-col items-center">
          {children}
        </main>

      </body>
    </html>
  );
}
```

Initialize state and callback function

Use theme state as CSS class on body

Pass state and callback down the component tree

Header

```tsx
import Link from 'next/link';
import { ThemeToggleButton } from './ThemeToggleButton';

type Props = {
  theme: string;
  toggleTheme: () => void;
};

const Header = ({ theme, toggleTheme }: Props) => {
  return (
    <header className="...">
      <a className="...">
        Courses
      </a>
      <nav className="...">
        ...

        <ThemeToggleButton theme={theme} toggleTheme={toggleTheme} />
      </nav>
    </header>
  );
};

export default Header;
```

Header only needs these props in order to pass them to ThemeToggleButton

ThemeToggleButton

```tsx
'use client';

type ButtonProps = {
  theme: string;
  toggleTheme: () => void;
};

export const ThemeToggleButton = ({ theme, toggleTheme }: ButtonProps) => {
  return (
    <button
      onClick={toggleTheme}
      className="ml-6 rounded bg-blue-500 px-4 py-2 font-bold text-white">
      {theme === 'light' ? 'Dark' : 'Light'} theme
    </button>
  );
};
```

Notify Header that theme needs to be switched.

Conclusion

- This implementation works, but demonstrates a bad coding practice called "**Prop drilling":**
  - A prop needs to be passed through several layers of components to reach a nested child component that needs the prop.
- Problems:
  - "Dirty" components: Header is required to accept Props it doesn't need.
  - Boilerplate code: we need to define the "theme" props in multiple places.
  - Layout.tsx needs to be a client component: we need the theme state at the root level.
  - Maintainability:
    - What if we decide to rename "theme" to "colorMode"? → changes in multiple files
    - What if we decide to add a NavigationMenu between Header and ThemeToggleButton? → Pass props down another layer

Solution

- We need a mechanism to “teleport” data from the “Provider” (Layout) to the “Consumer” (ThemeToggleButton), without bothering the “middleman” (Header).
- Enter **React Context**

React Context

- With React Context, the Provider component essentially *creates* a “box”.
- Any component that is placed *inside* this box can access the shared data within that box.
- Works only in client components because it relies on React hooks and state management, which are inherently features not supported by stateless Server Components.

React Context

1. Create the box
   - Define a box for specific data (e.g. theme data)
   - *createContext*
2. Fill the box and wrap around child components
   - Fill it with data (e.g. state & functions)
   - Wrap it with *Context.Provider*
3. Use data from the box
   - Any child component can read data from the box
   - *useContext*

Create Context

```tsx
'use client';

import { createContext, useState, ReactNode } from 'react';

export type ThemeContextType = {
  theme: string;
  toggleTheme: () => void;
};

export const ThemeContext = createContext<ThemeContextType | undefined>(
  undefined
);

export const ThemeProvider = ({ children }: { children: ReactNode }) => {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(theme === 'light' ? 'dark' : 'light');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <div className={theme}>{children}</div>
    </ThemeContext.Provider>
  );
};
```

Because we need state here.

Define the type of data in the box.

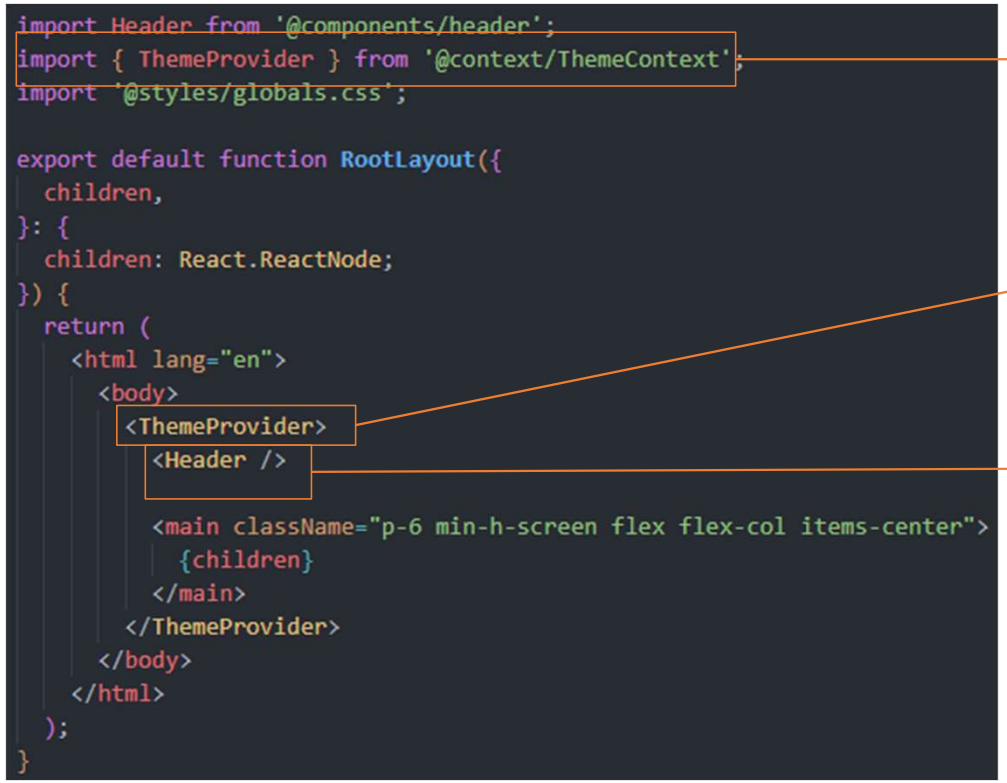
Create the box.

A wrapper div where the styling is applied.

Wraps the children to provide access to the theme data (e.g., Header is a child).

context/ThemeContext.tsx:

Wrap Context Provider in Layout



```tsx
import Header from '@components/header';
import { ThemeProvider } from '@context/ThemeContext';
import '@styles/globals.css';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <ThemeProvider>
          <Header />

          <main className="p-6 min-h-screen flex flex-col items-center">
            {children}
          </main>
        </ThemeProvider>
      </body>
    </html>
  );
}
```

Import our Theme context provider

Wrap the Context provider around child components

No more “prop drilling” in Header.

Header

```tsx
'use client';

import Link from 'next/link';
import { ThemeToggleButton } from './ThemeToggleButton';

export const Header = () => {
  return (
    <header>
      <nav>
        <Link href="/">Home</Link>
        ...

        <ThemeToggleButton />

      </nav>
    </header>
  );
};
```

No props anymore.

No more “prop drilling” in ThemeToggleButton.

ThemeToggleButton: Consume data

```
'use client';

import { useContext } from 'react';
import { ThemeContext } from '@context/ThemeContext';

export const ThemeToggleButton = () => {

  const context = useContext(ThemeContext);

  if (context === undefined) {
    throw new Error('ThemeToggleButton must be used within a ThemeProvider');
  }

  const { theme, toggleTheme } = context;

  return (
    <button onClick={toggleTheme} className="...">
      Switch to {theme === 'light' ? 'dark' : 'light'}
    </button>
  );
};
```

Use React’s context hook to get the current ThemeContext.

If, for some reason, we forgot to wrap the ThemeContext provider

Read data from context

Use context data as if it were props

How Next.js renders

1. Next.js renders server components (RootLayout) into the **React Server Component (RSC) Payload** (a JSON-like binary format).
2. Client Components (ThemeProvider) are marked as "**placeholders**" within this payload.
3. Server-side **pre-render**: render static HTML for a first, fast preview.
4. Client-side (browser): React uses the RSC Payload to build the **component tree**.
5. **Hydration**: activates the client components (JavaScript, e.g. ThemeProvider).
6. Server-side rendered children are **plugged** in the client components (ThemeProvider).

Next steps

- We’ve learned to use React Context for a simple ThemeContext example.
- We’re going to apply this pattern to another common “problem”: Authentication.